

Internal Operations AI Agent

Study Case AI Engineer - Dokumentasi Project

Nama: Fauzi Rahmat Febrian Tanggal: Juli 15 2026

1. Deskripsi Singkat Project

Internal Operations AI Agent adalah prototype AI agent untuk kebutuhan automation internal perusahaan. Agent ini menerima pertanyaan atau instruksi dalam bahasa natural, mengidentifikasi intent-nya, memilih tool yang sesuai, mengambil informasi dari knowledge base bila diperlukan, lalu menghasilkan output terstruktur (JSON) berisi jawaban, tingkat keyakinan, dan rekomendasi aksi lanjutan.

Agent mendukung 5 intent: menjawab pertanyaan dari knowledge base (knowledge_question), triase tiket masalah (ticket_triage), pembuatan task simulasi (create_task), ringkasan dokumentasi lintas-dokumen (summarize_request), dan penolakan yang jujur ketika informasi tidak tersedia (cannot_answer).

2. Tech Stack yang Digunakan

Komponen	Teknologi	Alasan
Bahasa	Python 3.10+	-
LLM	Google Gemini API (google-genai SDK)	Klasifikasi intent, ekstraksi task, ringkasan
Retrieval	scikit-learn (TF-IDF + cosine similarity)	Rule-based, deterministik, tanpa API call
Validasi output	Pydantic	Memaksa Gemini mengembalikan JSON terstruktur & tervalidasi
Data	pandas + CSV	knowledge_base.csv, sample_requests.csv
Konfigurasi	python-dotenv	Memuat GEMINI_API_KEY dari .env
Interface	CLI (main.py) & Streamlit (streamlit_app.py)	Dua opsi interface sesuai brief

3. Struktur Repositori

```
internal-ops-ai-agent/  
|  
|-- data/  
|   |-- knowledge_base.csv    # 10 dokumen KB  
|   `-- sample_requests.csv   # 12 test request  
|  
|-- tools.py                  # 5 tools (3 wajib + 2 optional)  
|-- agent.py                  # run_agent() - orkestrasi utama  
|-- main.py                   # CLI interface  
|-- streamlit_app.py          # Web chat interface (opsional)  
|-- run_evaluation.py          # Script evaluasi otomatis (checkpoint/resume)  
|  
|-- AGENT_DESIGN.md           # Task 1 - diagram alur + 5 jawaban wajib  
|-- PROMPT_ENGINEERING.md     # Task 4 - prompt utama + penjelasan  
arsitektur  
|-- evaluation.md              # Task 6 - hasil evaluasi nyata  
|-- README.md                  # Dokumentasi lengkap (versi markdown)  
|  
|-- requirements.txt  
|-- .env.example  
`-- .gitignore
```

4. Sumber Link Repository dan Hosting

Github Link Repository

<https://github.com/FauziRahmatFebrian/Internal-Operations-AI-Agent>

web chat hosting (Streamlit):

<https://operations-ai-agent.streamlit.app/>

5. Cara Menjalankan Project

Instalasi dependency dan konfigurasi API key:

```
pip install -r requirements.txt
cp .env.example .env
# lalu isi GEMINI_API_KEY asli di dalam .env
```

Menjalankan lewat CLI:

```
python main.py
```

Menjalankan lewat web chat (Streamlit):

```
streamlit run streamlit_app.py
```

Menjalankan lewat web chat hosting (Streamlit):

<https://operations-ai-agent.streamlit.app/>

Test tiap tool secara terisolasi (tanpa perlu jalankan seluruh agent):

```
python tools.py
```

Menjalankan evaluasi otomatis (hasilnya menulis ulang evaluation.md):

```
python run_evaluation.py
python run_evaluation.py --limit 8 # opsional, hemat kuota
```

Catatan model: gemini-2.5-flash sudah tidak tersedia untuk akun/project baru per Juli 2026. Default di .env.example diarahkan ke gemini-3.1-flash-lite. Jika model itu juga tidak tersedia, ganti GEMINI_MODEL di .env ke gemini-flash-latest.

6. Penjelasan Agent Workflow

```
User Input -> Intent Detection -> Tool Selection -> Tool Execution
-> Response Generation -> Structured Output -> Recommended Action
```

- Intent Detection: `classify_request()` (satu panggilan Gemini, output dipaksa JSON lewat Pydantic schema) mengembalikan intent, category, priority, confidence sekaligus.
- Intent yang didukung: `knowledge_question`, `ticket_triage`, `create_task`, `summarize_request`, `cannot_answer`.
- Tool selection: percabangan deterministik di Python (`agent.py`), dikunci ke intent - bukan LLM function-calling. Ini keputusan desain sadar untuk evaluability dalam waktu terbatas (lihat bagian 10).
- Menangani kasus tidak bisa dijawab: `KB_SCORE_THRESHOLD` (0.15) di `agent.py` menjadi gerbang - jika skor TF-IDF dokumen terbaik di bawah ambang ini, agent langsung mengembalikan pesan penolakan tetap tanpa memanggil LLM untuk merumuskan jawaban.
- Menghindari jawaban di luar knowledge base: untuk `knowledge_question/ticket_triage`, field answer diambil verbatim dari kolom content di `knowledge_base.csv` - LLM tidak pernah diminta merumuskan ulang teks jawabannya.

7. Daftar Tools yang Dibuat

Tool	Tipe	Fungsi
<code>search_knowledge_base(query)</code>	Rule-based (TF-IDF)	Mengambil dokumen KB paling relevan
<code>classify_request(user_input)</code>	LLM (Gemini + Pydantic)	Intent, category, priority, confidence
<code>create_task_object(...)</code>	LLM (Gemini + Pydantic)	Simulasi task object (tidak dikirim ke sistem nyata)
<code>summarize_text(query, documents)</code>	LLM (Gemini, grounded)	Merangkum >1 dokumen KB untuk <code>summarize_request</code>
<code>validate_json_output(output)</code>	Rule-based	Gerbang QA struktural sebelum respons dikembalikan

3 tool pertama wajib sesuai brief; 2 tool terakhir (`summarize_text`, `validate_json_output`) adalah tambahan optional.

8. Contoh Input dan Output CLI

Contoh berikut adalah hasil eksekusi nyata terhadap Gemini API lewat main.py (bukan disimulasikan).

Input:

```
Create a follow-up task for the finance team because the customer has not received the invoice.
```

Output:

```
{
  "user_input": "Create a follow-up task for the finance team because the customer has not received the invoice.",
  "intent": "create_task",
  "tools_used": ["classify_request", "create_task_object"],
  "result": {
    "task": {
      "title": "Customer missing invoice",
      "assigned_team": "Finance",
      "priority": "Medium",
      "description": "Customer has not received the invoice and requires a follow-up from the finance team."
    },
    "status": "simulated"
  },
  "confidence": "high",
  "recommended_action": "Review and create task in the internal system"
}
```

Contoh SS

```

PS C:\Data Latihan\test_magang> python main.py
Internal Operations AI Agent (type 'exit' to quit)

> Create a follow-up task for the finance team because the customer has not received the invoice.
{
  "user_input": "Create a follow-up task for the finance team because the customer has not received the invoice.",
  "intent": "create_task",
  "tools_used": [
    "classify_request",
    "create_task_object"
  ],
  "result": {
    "task": {
      "title": "Customer missing invoice",
      "assigned_team": "Finance",
      "priority": "Medium",
      "description": "A customer has not received their invoice and requires a follow-up from the finance team."
    },
    "status": "simulated"
  },
  "confidence": "high",
  "recommended_action": "Review and create task in the internal system"
}

```

Input:

Create a follow-up task for the finance team because the customer has not received the invoice.

Output:

```

{
  "user_input": "Can you tell me what salary my coworker is earning?",
  "intent": "cannot_answer",
  "tools_used": ["classify_request"],
  "result": {
    "answer": "I cannot answer this question based on the available knowledge base.",
    "sources": []
  },
  "confidence": "low",
  "recommended_action": "Escalate to human support"
}

```

Contoh SS

```
> Can you tell me what salary my coworker is earning?
{
  "user_input": "Can you tell me what salary my coworker is earning?",
  "intent": "cannot_answer",
  "tools_used": [
    "classify_request"
  ],
  "result": {
    "answer": "I cannot answer this question based on the available knowledge base.",
    "sources": []
  },
  "confidence": "low",
  "recommended_action": "Escalate to human support"
}
```

9. Contoh Input dan Output Streamlit

Interface web chat (streamlit_app.py) memanggil run_agent() yang sama persis dengan CLI - hanya beda cara menampilkan hasilnya (bubble chat + JSON mentah di dalam expander).

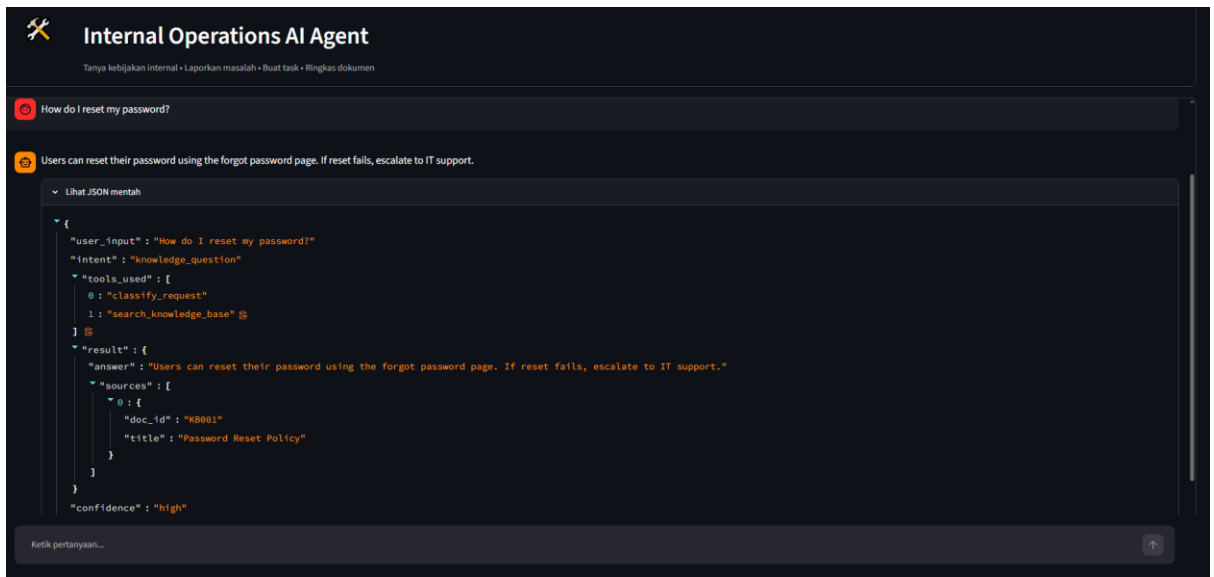
Input:

How do I reset my password?

Output:

```
{
  "user_input": "How do I reset my password?",
  "intent": "knowledge_question",
  "tools_used": ["classify_request", "search_knowledge_base"],
  "result": {
    "answer": "Users can reset their password using the forgot password page. If reset fails, escalate to IT support.",
    "sources": [{"doc_id": "KB001", "title": "Password Reset Policy"}]
  },
  "confidence": "high",
  "recommended_action": "Answer user directly"
}
```

Contoh ss



Input:

I cannot login after resetting my password.

Output:

```
{
  "user_input": "I cannot login after resetting my password.",
  "intent": "ticket_triage",
  "tools_used": ["classify_request", "search_knowledge_base"],
  "result": {
    "category": "Account Access",
    "priority": "High",
    "answer": "Users can reset their password using the forgot password page. If reset fails, escalate to IT support.",
    "sources": [{"doc_id": "KB001", "title": "Password Reset Policy"}]
  },
  "confidence": "high",
  "recommended_action": "Escalate to Account Access team if the user's issue persists"
}
```




10. Hasil Evaluasi

Ringkasan dari evaluation.md (12 test case, dijalankan nyata terhadap Gemini API lewat run_evaluation.py):

- Intent accuracy: $11/12 = 91.7\%$
- Tool selection accuracy: $12/12 = 100\%$ (heuristik - lihat catatan di bawah)

Satu case gagal: R009 - "My account got locked after too many failed logins, what do I do?" - diharapkan knowledge_question, agent memilih ticket_triage. Ini bukan random error: kalimat ini secara linguistik ambigu antara melaporkan insiden personal (ciri ticket_triage) dan bertanya panduan umum (ciri knowledge_question), dan definisi intent di prompt classify_request memang konsisten dengan pilihan LLM.

Analysis

1. Case mana yang berhasil?

11 dari 12 case (R001, R002, R003, R004, R005, R006, R007, R008, R010, R011, R012). Ini mencakup semua 5 intent yang didukung (knowledge_question, ticket_triage, create_task, summarize_request, cannot_answer), termasuk dua kasus cannot_answer (R004, R011) yang diklasifikasikan benar langsung oleh classify_request tanpa perlu masuk ke jalur fallback KB_SCORE_THRESHOLD di agent.py sama sekali - LLM sudah mengenali keduanya sebagai di luar domain operasional sejak awal.

2. Case mana yang gagal?

R009 - *"My account got locked after too many failed logins, what do I do?"* - expected knowledge_question, agent memprediksi ticket_triage.

3. Mengapa agent gagal?

Melaporkan sebuah kejadian yang sudah terjadi pada akun spesifik user ("got locked"), yang menyerupai ticket_triage, sekaligus meminta panduan umum ("what do I do?"), yang menyerupai knowledge_question. _CLASSIFY_SYSTEM_PROMPT di tools.py mendefinisikan ticket_triage sebagai "user is reporting a problem they are personally experiencing" - dan kalimat R009 memang secara literal melaporkan masalah yang sedang dialami, jadi klasifikasi LLM konsisten dengan instruksi yang diberikan, hanya berbeda dari label yang saya tetapkan saat membuat dataset. Ini bukan random error, tapi ambiguitas batas definisi intent itu sendiri.

4. Bagaimana cara memperbaiki agent?

Perbaikan paling langsung adalah menambahkan aturan eksplisit di _CLASSIFY_SYSTEM_PROMPT untuk membedakan dua pola ini secara lebih jelas - misalnya: "jika user meminta instruksi/prosedur untuk menangani situasi yang sudah terjadi, ini ticket_triage; jika user bertanya tentang kebijakan/prosedur secara umum tanpa melaporkan insiden pribadi, ini knowledge_question" - idealnya disertai 1-2 contoh (few-shot) khusus untuk pola "X terjadi, what do I do?" supaya modelnya konsisten. Perbaikan kedua: metrik *tool selection accuracy* saat ini tidak bisa mendeteksi kasus seperti ini karena knowledge_question dan ticket_triage kebetulan memakai tool set yang identik (classify_request + search_knowledge_base) - metrik ini perlu dianggap sebagai sinyal tambahan, bukan pengganti review manual pada baris yang gagal di kolom intent.

5. Apa risiko hallucination dari agent ini?

Untuk jalur knowledge_question dan ticket_triage, risikonya rendah secara struktural karena answer diambil verbatim dari content di knowledge_base.csv (lihat _answer_from_kb() di agent.py) - LLM tidak pernah diminta merumuskan ulang teks jawabannya, jadi tidak ada celah untuk menambahkan detail yang tidak ada di KB pada teks jawaban itu sendiri. Risiko yang tersisa: (a) classify_request bisa salah intent

seperti R009, yang mengubah `category/priority/ recommended_action` walau bukan `answer`-nya; (b) `create_task_object` dan `summarize_text` memang menghasilkan teks baru dari LLM (bukan verbatim), sehingga secara teori masih bisa menambahkan detail yang sedikit meleset dari input asli, walau dibatasi instruksi anti-fabrikasi di prompt masing-masing - risiko ini tidak sepenuhnya nol seperti jalur verbatim, dan belum ada test case yang secara khusus menguji seberapa sering `summarize_text` "menambahkan" detail (kelemahan evaluasi saat ini - baru diuji 1 kasus, R010).

6. Bagaimana agent menangani informasi yang tidak tersedia?

`KB_SCORE_THRESHOLD` (0.15) di `agent.py` menjadi gerbang: kalau skor TF-IDF dokumen terbaik di bawah ambang ini, agent mengembalikan pesan penolakan tetap tanpa memanggil LLM untuk merumuskan jawaban, dan mereklasifikasi intent menjadi `cannot_answer`. Hasil evaluasi ini menunjukkan mekanisme itu bahkan tidak selalu diperlukan - R004 dan R011 sudah dikenali sebagai `cannot_answer` langsung oleh `classify_request` di langkah pertama, sebelum `search_knowledge_base` sempat dipanggil sama sekali (`tools_used` keduanya cuma `["classify_request"]`).

Catatan metrik: tool selection accuracy 100% di sini termasuk kasus R009 yang sebenarnya salah intent - metrik ini kebetulan bernilai "Yes" karena `knowledge_question` dan `ticket_triage` kebetulan memakai tool set yang identik. Ini heuristik, bukan ground truth - detail lengkap ada di `evaluation.md`.

11. Catatan Singkat tentang Asumsi dan Keterbatasan

Asumsi:

- Rule-based tool selection (Python `if/elif` berdasarkan intent) dipilih sebagai pengganti LLM function-calling, dengan pertimbangan evaluability dalam waktu terbatas 2 hari - bukan karena LLM function-calling tidak memungkinkan.
- Dataset `knowledge_base.csv` dan `sample_requests.csv` adalah data dummy buatan sendiri sesuai izin brief, bukan data internal perusahaan sungguhan.

Keterbatasan:

- Ambiguitas intent pada kalimat "insiden + minta panduan" (lihat R009) - prompt `classify_request` belum punya aturan eksplisit/few-shot untuk pola ini.

- Dua dari enam kategori KB (Product Question, General Inquiry) belum pernah teruji di 12 baris sample_requests.csv yang dievaluasi.
- summarize_text baru diuji 1 kali (R010) - belum cukup data untuk menyimpulkan konsistensinya menghindari fabrikasi saat merangkum >1 dokumen.
- Retrieval TF-IDF murni leksikal - tidak menangkap kemiripan makna/parafrase seperti embedding.
- KB_SCORE_THRESHOLD (0.15) adalah nilai heuristik dari observasi manual, belum di-tuning dengan validation set sistematis.
- Kuota free-tier Gemini API membatasi kecepatan testing selama development.

12. Ide Pengembangan Selanjutnya

- Tambahkan aturan/few-shot example di prompt classify_request khusus untuk pola "insiden + minta panduan" (kasus R009).
- Perluas sample_requests.csv untuk menutup 2 kategori yang belum teruji (Product Question, General Inquiry).
- Ganti retrieval dari TF-IDF ke embedding-based similarity untuk menangkap kemiripan makna, bukan cuma leksikal.
- Eksperimen KB_SCORE_THRESHOLD secara sistematis dengan validation set yang lebih besar.
- Bandingkan arsitektur saat ini dengan pendekatan LLM function-calling murni menggunakan types.Tool di Gemini API.
- Tambahkan escalate_to_human(reason) sebagai tool eksplisit yang bisa dipanggil, bukan cuma string di recommended_action.
- Tambahkan memori percakapan multi-turn (saat ini setiap run_agent() call independen).
- Endpoint FastAPI sebagai alternatif interface untuk integrasi dengan sistem lain.